

Lab 2- Resource

THEORY: JavaFX Basics

1. **Stage** → The main window (think of it as a "frame" or "screen")
2. **Scene** → The content inside the window
3. **Node** → Anything inside the scene (Button, Label, TextField, etc.)
4. **Layout Pane** → Organizes nodes (e.g., `StackPane`, `VBox`, `HBox`)
5. **Event Handling** → Reacting to user actions (click, input, etc.)

CODE: Hello JavaFX

Step 1: Create JavaFX Project in NetBeans

1. **File** → **New Project** → **Java with Ant** → **JavaFX Application**
2. Project Name: `Lab2`
3. Click **Finish**

NetBeans will create a file like `Lab2.java` for you.

Step 2: Replace Code With This

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.layout.StackPane;
import javafx.stage.Stage;

public class Lab2 extends Application {

    @Override
    public void start(Stage primaryStage) {
        // Create a Button
        Button btn = new Button("Click Me");

        // Event handling
        btn.setOnAction(e -> {
            System.out.println("Button was clicked!");
        });

        // Layout
        StackPane root = new StackPane();
```

```
        root.getChildren().add(btn);

        // Scene
        Scene scene = new Scene(root, 400, 300);

        // Stage
        primaryStage.setTitle("Hello JavaFX");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

WHAT HAPPENS

1. Window opens (Stage)
2. Scene with a button appears
3. Clicking the button prints message in **NetBeans output console**

EXPLANATION

Component	What it does
Application	Base class for JavaFX
start(Stage)	Main window setup
Button	Interactive element
StackPane	Layout that centers everything
setOnAction	Event handler for click
Scene	Holds all UI components
primaryStage.show()	Displays window

PRACTICE TASK

1. Change button text → "Press Me"
2. Add **another button** below the first using VBox layout

3. Print different messages for each button click

NOTES

- Every **JavaFX app must extend Application**
- `launch(args)` → starts JavaFX runtime
- You **cannot run JavaFX code like normal Java class** (must use `Application`)

THEORY: Layout Panes

In JavaFX, **layouts** arrange your UI components (Nodes). Some common ones:

Layout	How it works
StackPane	Stacks nodes on top of each other (centered by default)
VBox	Vertical arrangement (top → bottom)
HBox	Horizontal arrangement (left → right)
BorderPane	Places nodes in top, bottom, left, right, center
GridPane	Table-like grid arrangement (row, column)

For multiple buttons or labels, **VBox** or **HBox** is easiest.

CODE: Multiple Buttons with VBox

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

public class Lab2 extends Application {

    @Override
    public void start(Stage primaryStage) {
        // Create Buttons
        Button btn1 = new Button("Click Me");
        Button btn2 = new Button("Press Me");
```

```

// Event handling
btn1.setOnAction(e -> System.out.println("Button 1 clicked!"));
btn2.setOnAction(e -> System.out.println("Button 2 clicked!"));

// Layout
VBox root = new VBox(10); // 10 px spacing
root.getChildren().addAll(btn1, btn2);

// Scene
Scene scene = new Scene(root, 400, 300);

// Stage
primaryStage.setTitle("Multiple Buttons Example");
primaryStage.setScene(scene);
primaryStage.show();
}

public static void main(String[] args) {
    launch(args);
}
}

```

WHAT THIS DOES

- `VBox(10)` → vertical layout with 10 pixels spacing
- `addAll(btn1, btn2)` → adds multiple node
- Clicking each button prints a **different message***

PRACTICE TASK

1. Add a **Label** above the buttons with text `"Welcome!"`
2. Clicking **Button 1** changes the label to `"Hello!"`
3. Clicking **Button 2** changes the label to `"Goodbye!"`

This introduces **real interaction**, which is the core of JavaFX apps.

CODE: Interactive Label Example

```

import javafx.application.Application;
import javafx.scene.Scene;

```

```

import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

public class Lab2 extends Application {

    @Override
    public void start(Stage primaryStage) {
        // Create Label
        Label label = new Label("Welcome!");

        // Create Buttons
        Button btn1 = new Button("Click Me");
        Button btn2 = new Button("Press Me");

        // Event Handling
        btn1.setOnAction(e -> label.setText("Hello!")); // change label text
        btn2.setOnAction(e -> label.setText("Goodbye!")); // change label text

        // Layout
        VBox root = new VBox(10); // 10 px spacing
        root.getChildren().addAll(label, btn1, btn2);

        // Scene
        Scene scene = new Scene(root, 400, 300);

        // Stage
        primaryStage.setTitle("Interactive Label Example");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}

```

WHAT'S HAPPENING

1. **Label** → Displays text at the top
2. **btn1 and btn2** → Each button has its **own event handler**
3. **label.setText("...")** → Updates UI dynamically
4. **Layout** → **VBox(10)** arranges Label and Buttons vertically

Now clicking buttons **changes the label text** in real time.

CODE: TextField Example

```
package hellofx;

import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.control.TextField;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

public class Lab2 extends Application {

    @Override
    public void start(Stage primaryStage) {
        // Create Label
        Label label = new Label("Welcome!");

        // Create TextField
        TextField textField = new TextField();
        textField.setPromptText("Type something here...");

        // Create Submit Button
        Button submitBtn = new Button("Submit");

        // Event Handling
        submitBtn.setOnAction(e -> {
            String userInput = textField.getText(); // get user input
            label.setText("User typed: " + userInput);
        });

        // Layout
        VBox root = new VBox(10); // 10 px spacing
        root.getChildren().addAll(label, textField, submitBtn);

        // Scene
        Scene scene = new Scene(root, 400, 300);

        // Stage
        primaryStage.setTitle("TextField Interaction Example");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

```
}  
}
```

WHAT'S HAPPENING

1. `TextField` → User types text
2. `Button Submit` → Click triggers event handler
3. `textField.getText()` → Retrieves user input
4. `label.setText(...)` → Updates UI dynamically

Now you have a **fully interactive mini form**.

KEY CONCEPTS TO REMEMBER

- `TextField.getText()` → retrieves user input
- `Label.setText(...)` → updates output in real-time
- `Button.setOnAction(...)` → responds to clicks
- `Layout ( VBox )` manages spacing automaticall